UTEC
Universidad Tecnológica

Patrón Facade

Patrón Facade: Simplificación de Interfaces Complejas

Descripción General

El patrón Facade es un patrón estructural que proporciona una interfaz simplificada y unificada a un conjunto de interfaces de un subsistema o un sistema complejo. La idea principal es ocultar la complejidad del sistema proporcionando una interfaz más fácil de usar y accesible, lo que facilita a los clientes interactuar con el sistema sin necesidad de conocer los detalles internos de su implementación.

Este patrón es particularmente útil cuando un sistema es complejo o difícil de entender, y se quiere proporcionar una interfaz de uso sencillo que permita realizar operaciones comunes de manera más directa y menos propensa a errores.

Aplicaciones Comunes

- **Sistemas de bibliotecas complejas**: Donde se quiere encapsular la complejidad de una biblioteca en una interfaz simple.
- **Sistemas de grandes aplicaciones empresariales**: Donde diferentes subsistemas pueden ser encapsulados para reducir la complejidad.
- **Sistemas legacy**: Al integrar sistemas antiguos con nuevas aplicaciones, el patrón Facade puede ser utilizado para simplificar la interacción.

Ventajas

- **Simplificación**: Ofrece una interfaz fácil de usar, ocultando la complejidad de las interacciones internas.
- **Desacoplamiento**: Reduce la dependencia entre los clientes y los subsistemas, facilitando cambios en la implementación interna sin afectar a los clientes.
- **Mantenimiento**: Facilita el mantenimiento y la evolución del sistema al centralizar las interacciones comunes en un solo lugar.

Desventajas

- **Posible sobrefuncionalidad**: La interfaz del Facade puede volverse demasiado amplia, incluyendo funcionalidades que no todos los clientes necesitan.
- **Menos flexibilidad**: Puede limitar el acceso a las funcionalidades completas de los subsistemas para los usuarios que necesitan operaciones más específicas.

UTEC
Universidad Tecnológica

Ejemplo en Java

Supongamos que estamos desarrollando un sistema para la gestión de un hogar inteligente, donde hay varios subsistemas como luces, climatización, y sistema de seguridad. El patrón Facade nos ayudará a simplificar la interacción con estos subsistemas.

```
// Subsistemas
class Luces {
    public void encender() {
        System.out.println("Luces encendidas.");
    }

    public void apagar() {
        System.out.println("Luces apagadas.");
    }
}

class Climatizacion {
    public void encender() {
        System.out.println("Climatización encendida.");
    }

    public void ajustarTemperatura(int grados) {
        System.out.println("Temperatura ajustada a " + grados + "
grados.");
    }

    public void apagar() {
        System.out.println("Climatización apagada.");
    }
}

class Seguridad {
    public void activar() {
        System.out.println("Sistema de seguridad activado.");
    }

    public void desactivar() {
        System.out.println("Sistema de seguridad desactivado.");
    }
}
```

UTEC
Universidad Tecnológica

```java
// Facade
class HomeFacade {
    private Luces luces;
    private Climatizacion climatizacion;
    private Seguridad seguridad;

    public HomeFacade() {
        this.luces = new Luces();
        this.climatizacion = new Climatizacion();
        this.seguridad = new Seguridad();
    }

    public void llegarACasa() {
        luces.encender();
        climatizacion.encender();
        climatizacion.ajustarTemperatura(22);
        seguridad.desactivar();
        System.out.println("Bienvenido a casa.");
    }

    public void salirDeCasa() {
        luces.apagar();
        climatizacion.apagar();
        seguridad.activar();
        System.out.println("Hasta luego. Su hogar está seguro.");
    }
}

// Uso del patrón Facade
public class FacadePatternExample {
    public static void main(String[] args) {
        HomeFacade homeFacade = new HomeFacade();

        // Llegar a casa
        homeFacade.llegarACasa();

        // Salir de casa
        homeFacade.salirDeCasa();
    }
}
```

UTEC
Universidad Tecnológica

Explicación del Ejemplo

En este ejemplo, hemos creado un sistema de hogar inteligente con tres subsistemas: `Luces`, `Climatizacion`, y `Seguridad`. Cada uno de estos subsistemas tiene su propia lógica y métodos. La clase `HomeFacade` actúa como una fachada que proporciona métodos simples para acciones comunes, como `llegarACasa` y `salirDeCasa`. Los métodos de la fachada simplifican la interacción con los subsistemas, de manera que el usuario final solo necesita invocar un único método en lugar de interactuar con cada subsistema por separado.

Conclusión

El patrón Facade es esencial cuando se desea simplificar la interacción con sistemas complejos. Proporciona una interfaz limpia y simple, que oculta la complejidad interna de los subsistemas, permitiendo a los usuarios realizar operaciones comunes de manera fácil y eficiente. Este patrón es muy útil en grandes sistemas empresariales, bibliotecas complejas, y cuando se integra con sistemas legacy.

Ejemplo Práctico: envío de un correo electrónico

Para enviar un correo electrónico en Java, puedes utilizar la API **JavaMail**. Este paquete te permite enviar correos electrónicos desde tu aplicación de forma sencilla. Primero vamos a ver los pasos básicos para enviar un correo, y después cómo podrías aplicar un patrón de diseño para estructurar este proceso.

1. Envío de correo electrónico usando JavaMail

Primero, debes incluir la dependencia de JavaMail en tu proyecto (con **Maven**):

```
<dependency>
    <groupId>javax.mail</groupId>
    <artifactId>javax.mail-api</artifactId>
    <version>1.6.2</version>
</dependency>
<dependency>
    <groupId>com.sun.mail</groupId>
    <artifactId>javax.mail</artifactId>
    <version>1.6.2</version>
</dependency>
```

UTEC
Universidad Tecnológica

2. Configuración y envío del correo

El siguiente código te muestra cómo enviar un correo electrónico usando JavaMail.

```
import java.util.Properties;
import javax.mail.*;
import javax.mail.internet.InternetAddress;
import javax.mail.internet.MimeMessage;

public class EmailSender {

    public static void enviarCorreo(String destinatario, String asunto,
String cuerpo) {
        // Configuración del servidor de correo (SMTP)
        Properties props = new Properties();
        props.put("mail.smtp.auth", "true");
        props.put("mail.smtp.starttls.enable", "true");
        props.put("mail.smtp.host", "smtp.gmail.com");  // Servidor SMTP
        props.put("mail.smtp.port", "587");  // Puerto SMTP

        // Autenticación
        String remitente = "tuemail@gmail.com"; // Email del remitente
        String password = "tu_password"; // Contraseña del remitente

        Session session = Session.getInstance(props, new
javax.mail.Authenticator() {
            protected PasswordAuthentication getPasswordAuthentication()
{
                return new PasswordAuthentication(remitente, password);
            }
        });

        try {
            // Crear el mensaje
            Message message = new MimeMessage(session);
            message.setFrom(new InternetAddress(remitente));
            message.setRecipients(Message.RecipientType.TO,
InternetAddress.parse(destinatario));
            message.setSubject(asunto);
            message.setText(cuerpo);
```

UTEC
Universidad Tecnológica

```
            // Enviar el correo
            Transport.send(message);

            System.out.println("Correo enviado exitosamente!");

        } catch (MessagingException e) {
            e.printStackTrace();
        }
    }

    public static void main(String[] args) {
        enviarCorreo("destinatario@example.com", "Asunto de prueba",
"Este es el cuerpo del correo.");
    }
}
```

Explicación del código

- **Properties**: Configura las propiedades de la conexión SMTP (servidor de correo saliente) que se va a usar para enviar el correo.
- **Session**: Crea una sesión para la autenticación con el servidor.
- **Message**: Se define el mensaje del correo con su remitente, destinatario, asunto y cuerpo.
- **Transport**: Método encargado de enviar el correo.

3. Aplicación del patrón de diseño: Facade

El patrón de diseño **Facade** es ideal para este tipo de situaciones, donde quieres encapsular la complejidad de la interacción con varias clases o subsistemas en una sola interfaz sencilla. En este caso, podrías implementar un **Facade** para enviar correos electrónicos, ocultando la complejidad de la configuración y el envío de correos.

Implementación del patrón Facade para el envío de correos

```
public class EmailFacade {

    private String remitente;
    private String password;
```

UTEC
Universidad Tecnológica

```
    public EmailFacade(String remitente, String password) {
        this.remitente = remitente;
        this.password = password;
    }

    public void enviarCorreo(String destinatario, String asunto, String
cuerpo) {
        Properties props = new Properties();
        props.put("mail.smtp.auth", "true");
        props.put("mail.smtp.starttls.enable", "true");
        props.put("mail.smtp.host", "smtp.gmail.com");
        props.put("mail.smtp.port", "587");

        Session session = Session.getInstance(props, new
javax.mail.Authenticator() {
            protected PasswordAuthentication getPasswordAuthentication()
{
                return new PasswordAuthentication(remitente, password);
            }
        });

        try {
            Message message = new MimeMessage(session);
            message.setFrom(new InternetAddress(remitente));
            message.setRecipients(Message.RecipientType.TO,
InternetAddress.parse(destinatario));
            message.setSubject(asunto);
            message.setText(cuerpo);
            Transport.send(message);
            System.out.println("Correo enviado correctamente!");
        } catch (MessagingException e) {
            e.printStackTrace();
        }
    }
}
```

Uso del Facade

```
public class Main {
    public static void main(String[] args) {
```

UTEC
Universidad Tecnológica

```
        // Crear el objeto Facade
        EmailFacade emailFacade = new EmailFacade("tuemail@gmail.com",
"tu_password");

        // Enviar el correo
        emailFacade.enviarCorreo("destinatario@example.com", "Asunto",
"Cuerpo del correo");
    }
}
```